DATEVRA

ULTRA MASTER PROMPT

Complete Tinder-Style Member Experience Audit, Implementation & Verification

Five coordinated phases • Existing Supabase architecture • Real member data • Luxury brand preservation

Execution rule	Required behavior
Scope	Audit and implement all five phases in the existing Datevra member-facing application.
Execution mode	Implement sequentially in one coordinated project, then run full verification.
Production safety	Zero remote SQL/migrations, zero production data/settings changes, zero deploys, zero pushes.
Protected areas	Admin Panel, Datevra branding/design system, existing accounts and historical data.
Completion standard	Real end-to-end behavior and persistence—not appearance, mock data, or claimed success.

HOW TO USE THIS DOCUMENT

Upload this DOCX into the existing Datevra Google AI Studio project/workspace, then send the short instruction below. Do not paste only the filename.

Read this entire document and follow it as the authoritative master specification. First inspect the current workspace and reconcile this specification with the latest code and the audit findings already provided. Then implement Phases 1–5 sequentially in the existing application, with focused tests after each phase. Do not stop after Phase 1. Do not skip requirements. Do not make any production changes, run SQL, apply migrations, push code, or deploy. Preserve Datevra branding and the Admin Panel. At the end, provide the complete evidence-based implementation and verification report requested in this document.

1. ROLE, OBJECTIVE & EXECUTION MODE

Act as a senior full-stack engineer, React/TypeScript architect, Supabase security reviewer, UX engineer, accessibility specialist, and QA lead. You are working inside the existing Datevra application, not creating a new application.

Objective: deliver a coherent, production-quality, Tinder-inspired member experience using Datevra's own identity and existing backend architecture. Tinder is a reference only for familiar interaction patterns such as card-based discovery, swipe gestures, mutual-match feedback, and clear navigation. Do not copy Tinder's name, logo, wording, visual assets, distinctive colors, or proprietary screen design.

- Perform an initial read-only inspection of the current workspace before editing.
- Use the supplied audit and feature matrix as a starting point, not as unquestionable truth. Verify every important claim against current source code, migrations, types, policies, and actual call paths.
- Implement all five phases in order during this task. Do not wait for separate user approval between phases; the user explicitly authorizes the complete local implementation through this master prompt.
- If a feature cannot safely be completed because its backend contract, schema, policy, or required infrastructure is missing or unclear, do not fake it. Complete safe independent work, mark the blocked portion clearly, and explain the exact dependency.
- Make the smallest cohesive changes that achieve the requirements. Avoid broad refactors and unrelated cleanup.

2. NON-NEGOTIABLE SAFETY & PRESERVATION RULES

2.1 Local-only boundary

- Work only in the currently open Google AI Studio/local project workspace.
- Do not connect to or modify the remote production Supabase project as part of this task.
- Do not execute SQL in Supabase SQL Editor, run migrations remotely, change production rows, alter production Auth/Storage/RLS settings, create production users, or modify secrets/DNS/hosting.
- Do not deploy, publish, commit, push, merge, or create a pull request.
- If schema changes are required, create a migration file locally only. Include preflight checks, constraints, rollback considerations, and a clear explanation. Never execute it.
- Never request, print, expose, hardcode, or place credentials, API keys, service-role keys, access tokens, or secrets in browser code, prompts, reports, screenshots, or logs.
- Never use a service-role key in client-side code. Respect existing environment-variable boundaries.

2.2 Protected product areas

- Do not redesign, refactor, or change the Admin Panel, admin routes, admin permissions, admin queries, admin workflows, or admin visual layout.
- Preserve Datevra name, logo, luxury identity, color palette, typography, gradients, spacing, borders, shadows, components, layout patterns, animation language, and existing branding.
- Do not change global design tokens or shared components in ways that alter unrelated pages. If a shared component must be extended, preserve all existing consumers and verify them.
- Do not remove or rename existing routes, deep links, tables, columns, RPCs, Edge Functions, storage buckets, or public interfaces without a demonstrated necessity and a safe compatibility plan.

2.3 Data, security and integrity

- Preserve all existing accounts, profiles, photos, likes, matches, conversations, messages, subscriptions, passes, payments, reports, blocks, and historical records.
- Never seed fake/demo users, fabricated likes, fake matches, sample conversations, invented compatibility scores, or simulated persistent success.
- Never weaken Row Level Security, authorization, ownership checks, age eligibility, block rules, verification restrictions, or privacy boundaries.
- Never expose private profile fields or sensitive answers to users who are not authorized to see them.
- All user actions must use real authenticated backend operations and truthful success/error states.
- Use existing tables, RPCs, triggers, Edge Functions, storage conventions, and data mappers wherever possible. Do not create parallel systems.
- Make mutations idempotent where repeated clicks, retries, concurrency, or network recovery could otherwise create duplicates.
- Do not silently overwrite existing values with empty strings, nulls, empty arrays, or defaults when a user edits only part of a profile.

3. VERIFIED STARTING CONTEXT — RECHECK BEFORE CODING

The following is the current supplied audit baseline. Verify it against the present workspace and report discrepancies before implementation.

Area / route	Supplied baseline	Required outcome
/register	Email/password/name/age/gender/interested-in/city; OTP verification; single-screen form.	Preserve working signup/OTP; guide new users into resumable onboarding.
/login	Email/password/forgot password; session resumption.	Preserve existing behavior and sessions.
/onboarding	Dedicated route currently missing.	Add six-step resumable onboarding.
/discover	Discovery RPC, ranking, swipe gestures; partially working.	Improve multi-photo card, gestures, locking, accessibility and states.
/likes	Incoming/outgoing likes; Like back and Pass.	Add useful profile preview and reliable response behavior.
/matches	Mutual matches, curated proposals, chat links, unmatch and compatibility.	Preserve and refine only where needed.
/chat	Matched 1-to-1 messaging, realtime, typing and read receipts.	Preserve data/realtime; improve mobile controls and overflow.
/profile	Profile editor and photo management.	Keep fields and validation in parity with onboarding.
/settings	Notifications, account deletion, preferences.	Preserve.
/privacy	Blocked users, online toggle, visibility controls.	Preserve.

4. PROFILE DATA & EXISTING BACKEND CONTRACTS

The supplied audit identifies these profile concepts in public.profiles and the existing TypeScript types/mappers. Confirm exact current names, nullability, enum values, and persistence mapping; do not assume the display labels equal database column names.

Concept	Supplied fields / values
Identity	id (uuid), email (private), firstName, age, gender (woman | man | nonbinary), city
Intent & story	bio, relationshipIntent (RELATIONSHIP_INTENTS), interestedIn (women | men | everyone)

Concept	Supplied fields / values
Personality	interests (text[]), hobbies (text[]), lifestyle (text[])
Media	photos (text[] URLs in profile-photos bucket)
Presence & security	online, lastActive, profileComplete (0–100), verified, isAdmin

- Use the existing profile mapper/types and profileCompleteness() calculation; do not invent a second completion formula.
- Use existing RELATIONSHIP_INTENTS and ALL_INTERESTS constants and the current tag/chip conventions.
- Use the existing uploadPhoto/deletePhoto pipeline and profile-photos bucket conventions.
- The supplied photo ownership convention is <authenticated_user_id>/<uuid>.<extension>; confirm current storage policies and implementation.
- The supplied discovery source is sb().rpc('discovery_profiles'); confirm the actual RPC contract and current filtering.
- The supplied compatibility function is computeCompatibility with real attributes; preserve explainability and never fabricate a score.
- The supplied like flow uses public.likes, mutual-like trigger handle_mutual_like(), ordered match pairs, conversations, unique constraints, and advisory locking in migrations 015/016. Confirm current schema/trigger behavior.
- The supplied chat system uses public.conversations and public.messages, room:<convId> and presence:<convId> realtime channels, and participant RLS. Confirm actual policy and subscription behavior.
- Treat all audit statements as leads. Do not alter backend assumptions without code/schema evidence.

5. MANDATORY INITIAL RECONCILIATION AUDIT

Before editing, inspect the full current project. Do not perform any remote calls or writes. Produce a concise internal implementation checklist and then proceed; do not stop for approval unless a safety-critical ambiguity makes further work unsafe.

- ■ Map all routes, route guards, authentication state transitions, registration/OTP callbacks, and member navigation.
- ■ Trace profile creation, retrieval, mapping, update, completeness calculation, and legacy-account behavior.
- ■ Trace photo upload, validation, progress, ordering, primary-photo selection, deletion, URL/path conversion, and storage ownership checks.
- ■ Trace discovery RPC/query, candidate filtering, ranking, exclusion lists, card rendering, swipe events, Like/Pass handlers, and full-profile modal.
- ■ Trace incoming/outgoing likes, Like back, Pass, mutual-match trigger, match lookup, duplicate handling, block/unmatch/report behavior.
- ■ Trace conversations, message sending, realtime subscriptions, typing, read receipts, unread counts, notification creation and cleanup.
- ■ Inspect RLS policies, SQL migrations, RPC definitions, Edge Function contracts, generated types, and any local schema snapshots available in the repository. Read-only only.
- ■ Inspect responsive layouts, navigation, shared components, accessibility patterns, tests, lint scripts, TypeScript settings, build scripts, and applet-specific compile/lint tools.
- ■ Identify any prior repair migrations/files in the workspace and avoid duplicating their purpose.

Create or update a local audit note with each feature classified as: Working / Partially working / UI-only / Broken / Missing / Blocked by backend. Record exact evidence (file paths, functions, route names, schema object names) rather than assumptions.

6. PHASE 1 — ACCOUNT CREATION, GUIDED ONBOARDING & PROFILE PARITY

6.1 Registration and authentication preservation

- Preserve existing email/password signup, email normalization (.trim().toLowerCase()), OTP verification, resend cooldown, duplicate-account handling, login links, password-reset links, and existing auth error handling.
- Preserve the existing 18+ eligibility check. Validate age at the appropriate boundary and never allow an underage member to bypass it through onboarding.
- Do not create duplicate Auth users or duplicate profiles. Handle the case where Auth succeeds but profile initialization is delayed or retried.
- After successful registration/OTP, route new members into onboarding only if their profile needs setup. Preserve safe session restoration.

6.2 Six-step onboarding

Create the dedicated member-facing route/page (suggested path /onboarding) using the current router and shared design system. Build six logical steps:

Step	Required fields and behavior
1. Basic Identity	First name, age (18+), gender, city. Reuse current enum/options and validation.
2. About You	Bio/introduction and headline only if a supported field exists. If headline has no schema field, do not invent one; either omit it or explain a local migration dependency.
3. Dating Intention	Relationship intent from RELATIONSHIP_INTENTS and interested-in preference from the existing supported field.
4. Interests & Hobbies	Selectable chips from ALL_INTERESTS and existing custom-tag behavior. Preserve values outside the visible preset list.
5. Photos	Upload through existing secure uploadPhoto pipeline; show progress/errors; support ordering and primary photo only if existing model/storage behavior supports it.
6. Summary & Review	Preview the member's own profile using existing profile card conventions; allow back/edit navigation before finishing.

- Show clear progress (for example, step count/progress bar) without changing the global design language.
- Allow back/next navigation; validate the current step before advancing and show field-level accessible errors.
- Persist each step safely using the existing authenticated profile update mechanism so refresh, sign-out/sign-in, or accidental navigation does not erase answers.
- On mount, load the authenticated member's current profile and initialize fields from saved values. Never initialize an existing profile to blank values.
- Do not mark the profile complete until the real profileCompleteness() rules say it is complete.
- Allow optional fields to remain optional. Do not force users to invent a bio, tags, or photos if existing product rules do not require them.
- Prevent duplicate saves and show accurate loading, retry, offline, and error states.
- If profile save fails, retain the entered values locally in the current page state and explain how to retry; do not falsely show completion.

6.3 Legacy and existing member behavior

- Members with completed profiles must bypass onboarding and proceed to their intended member route without friction.
- Incomplete legacy members may resume onboarding, but do not trap them or prevent access to /app, /discover, or /profile unless a current, documented product rule already requires it.
- Never force all existing users through the wizard on their next login.
- Do not overwrite profile data when a member skips a step or updates only one field.

6.4 Profile editor parity

- Update Profile.tsx only as needed so onboarding and profile editing use the same supported fields, constants, validation, tag conventions, photo logic, and real completion calculation.
- Avoid duplicate implementations by extracting/reusing narrowly scoped shared field components or helpers where this is safe.
- Keep profile editing backward compatible; preserve fields that are not currently visible in the editor.
- Add a truthful completion indicator based on profileCompleteness(), not a hardcoded percentage.

7. PHASE 2 — DISCOVERY DECK, MULTI-PHOTO CARDS & ACCESSIBLE SWIPING

7.1 Real candidate source and eligibility

- Continue using the existing discovery RPC/query and eligibility rules. Do not create a new discovery table or demo fallback.
- Respect self-exclusion, bidirectional blocks, hidden/deleted profiles, age/visibility preferences, verification/access rules, already-liked/passed/excluded profiles, and privacy restrictions as currently defined.
- Keep ranking deterministic and based only on real permitted profile attributes. Do not invent a compatibility percentage or silently change its formula.

7.2 Multi-photo card experience

- Upgrade the existing ProfileCard in src/pages/user/Discover.tsx (or the actual current component location after inspection).

- Render segmented progress indicators for the current candidate's actual photos.
- Provide tap zones: left 40% moves to the previous photo; right 60% moves to the next photo. Define clear first/last behavior (no accidental candidate change).
- Allow keyboard-accessible previous/next photo controls where appropriate; provide accessible labels and visible focus.
- Support swipe/tap without triggering page-level navigation or interfering with the card's horizontal Like/Pass drag gesture.
- Use smooth but restrained transitions consistent with Datevra's current motion style and honor prefers-reduced-motion.
- Handle zero photos, one photo, broken image URLs, loading and image failures gracefully without inventing replacement member photos.
- Reset photo index and progress when the active candidate changes.

7.3 Like/Pass interaction reliability

- Add a single action lock/busyAction guard to prevent rapid double submissions and overlapping Like/Pass requests.
- Disable or safely ignore repeated actions while a request is in flight; release the lock in a finally path.
- Use idempotent backend behavior and safe retry semantics. Do not optimistically claim a Like, Pass, or Match succeeded before the server confirms it.
- Prevent card gestures from causing duplicate action dispatches on touch devices.
- Preserve the existing Framer Motion drag behavior and LIKE/NOPE overlays where compatible.

7.4 Keyboard and accessibility

- Left Arrow = Pass; Right Arrow = Like; Space or Up Arrow = open full profile, only when focus is not inside an input, textarea, select, editable element, or dialog control.
- Provide accessible Like, Pass, and Full Profile buttons; keyboard shortcuts must not be the only way to use the feature.
- Ensure focus management and screen-reader announcements for candidate changes and action outcomes.

7.5 Filters and deck states

- Preserve existing city and age filters. Synchronize active UI state with actual query parameters/requests; never show a filter as active when it is not applied.
- Do not add cosmetic filters that do not affect the real backend query.
- Implement truthful loading, empty, no-more-candidates, offline, and error states. Provide retry where safe.
- Never substitute fake/demo profiles when discovery returns no results.

8. PHASE 3 — LIKES, MUTUAL MATCHES & COMPATIBILITY

8.1 Likes page

- Preserve real incoming and outgoing likes queries and existing tab semantics.
- Add an accessible full-profile preview/inspection action to incoming-like cards so a member can review the person before responding.
- Reuse the existing Full Profile modal/gallery and privacy rules; do not create a separate incompatible profile viewer.
- Ensure Like back and Pass actions use existing backend handlers and show real server-confirmed outcomes.
- Prevent double clicks, duplicate transitions, race conditions, and stale list state. Refresh or reconcile from authoritative backend state after mutations.

8.2 Match integrity

- Preserve the existing mutual-like trigger and backend match creation. Do not create matches from a client-side calculation.
- Respect unique constraints, ordered user pairs, advisory locking, and idempotent match lookup behavior already present.
- Verify duplicate Like requests, concurrent reciprocal Likes, repeated Like back, and network retry behavior.
- Preserve unmatch, block, report, and safety workflows. Do not re-match blocked users or bypass exclusion rules.
- Preserve match history and conversations unless existing product rules explicitly remove access.

8.3 Compatibility transparency

- Use the existing computeCompatibility() function and real input attributes.

- Keep the score explainable through actual factors such as shared interests, intent, age fit, city, lifestyle, and completeness only if those are truly part of the current implementation.
- Do not manufacture or hardcode scores or compatibility explanations.
- If the existing score cannot be verified or a factor is missing, disclose the limitation rather than fabricating it.

9. PHASE 4 — CHAT, NOTIFICATIONS & MEMBER NAVIGATION

9.1 Member navigation

- Refine src/components/layout.tsx or the actual current member layout component.
- Make the core member flows easy to reach: Discover, Likes, Matches, Chat, Profile.
- Add a Likes badge only when a real unread/new incoming-like count exists and the existing privacy/product rules permit it.
- Optimize mobile bottom navigation for narrow screens while preserving Datevra's design system and touch targets.
- Retain full desktop sidebar navigation and all existing routes, including Cafés and other member areas. Do not remove features merely to fit the dating flow.
- Avoid badge flicker, stale counts, repeated polling, and duplicate realtime listeners.

9.2 Chat mobile polish

- Preserve existing 1-to-1 matched messaging, message history, realtime channels, typing indicators, read receipts, and participant authorization.
- Improve mobile header layout and touch targets. Provide clear access to partner full profile and existing unmatch/block/report actions.
- Prevent horizontal overflow and ensure long names, message text, timestamps, and controls wrap or truncate accessibly.
- Preserve conversation switching/search and message history.
- Do not broaden who can read or send messages. Keep current matched-participant RLS rules.
- Ensure subscriptions are cleaned up on route change/unmount and do not multiply after reconnects.
- Handle send failures with truthful status and safe retry that cannot create duplicate messages.

9.3 Notifications

- Reuse existing notification tables/services/realtime channels if present.
- Only create or show notifications for real likes, matches, and messages according to current product rules.
- Do not add duplicate notification creation if database triggers or existing backend functions already do it.
- Do not invent notifications for events that the backend does not record.

10. PHASE 5 — RESPONSIVENESS, ACCESSIBILITY, SECURITY & QUALITY

10.1 Responsive verification

- Inspect and verify narrow mobile widths below 380px, standard mobile 390–430px, tablet, and desktop.
- Check onboarding steps, photo upload/reorder, discovery card and photo navigation, Likes, Matches, chat, navigation, profile editor, modals, and empty/error states.
- Respect safe-area insets, avoid clipped buttons/labels, and maintain readable text and usable touch targets (aim for at least 44×44 CSS px where practical).
- Verify portrait and landscape where the app supports both.

10.2 Accessibility

- Use semantic headings, buttons, labels, form associations, keyboard navigation, visible focus, and meaningful accessible names.
- Manage focus when opening/closing modals and restore focus to the invoking control.
- Announce important asynchronous status changes without overwhelming screen readers.
- Respect reduced motion and sufficient contrast within the existing palette; do not unilaterally recolor the brand.
- Ensure all core actions can be completed without drag gestures.

10.3 Performance and reliability

- Lazy-load images where appropriate and avoid loading every full-resolution photo at once.
- Prevent unnecessary rerenders and duplicate network requests without broad architectural rewrites.
- Check realtime subscription lifecycle, event deduplication, request cancellation, and stale async updates after unmount.
- Ensure loading states do not flash fake data or misleading completion states.

10.4 Security review

- Verify authorization checks exist on the server/database for profile edits, photo storage, Likes, Matches, chat and message access.
- Client-side checks are not a substitute for RLS or server authorization.
- Do not log passwords, tokens, private email fields, sensitive profile answers, or message contents unnecessarily.
- Do not weaken or bypass existing policies to make a UI feature work.
- If a policy or server contract is missing, mark the feature blocked and provide a local-only proposed migration or function change. Do not apply it remotely.

11. AUTOMATED TESTING & MANUAL VERIFICATION

Use the existing test framework and scripts. Do not delete or weaken tests to obtain a green result.

11.1 Required existing suites

- Run test_master_repairs_verification.ts and test_ultimate_master_repairs.ts through the repository's existing supported scripts.
- Run TypeScript checks, lint, application build, compile_applet, and lint_applet where available.
- If a script name differs or a tool is unavailable, report the exact command attempted and the limitation. Do not claim it passed.

11.2 New focused verification suite

Create test_tinder_ux_verification.ts (or the project-appropriate equivalent) covering:

- Onboarding step validation, 18+ validation, saved-value hydration, resumability, partial update preservation, and completed-member bypass.
- Profile completion derived from the existing real completeness function.
- Multi-photo previous/next navigation, index reset between candidates, zero/one-photo behavior, and card gesture boundaries.
- Busy action lock preventing duplicate Like/Pass dispatches and unlocking after success/failure.
- Like back idempotency and correct UI reconciliation after backend response.
- No client-created mutual match; match state comes from authoritative backend result.
- No fake/demo candidate fallback when discovery is empty or errors.
- Keyboard shortcuts ignored inside editable controls and modal contexts.
- Chat subscription cleanup and no duplicate event/message handling where testable.
- Legacy account routes and Admin Panel regression protection.

11.3 Manual test checklist

- New user signup → OTP verification → onboarding → review → member app.
- Refresh browser mid-onboarding → previously saved values restored.
- Existing complete member logs in → bypasses onboarding.
- Incomplete legacy member logs in → can resume without losing data or being trapped.
- Upload photo → progress/error handling → reorder/primary selection if supported → reload and verify persistence.
- Discovery card → navigate through all photos → next/previous candidate behavior remains correct.
- Like/Pass by touch, buttons and keyboard → no duplicate actions.
- Incoming Like → inspect full profile → Like back or Pass → server-confirmed result.
- Mutual Like → exactly one match/conversation according to backend design.
- Block/unmatch/report safeguards remain available and effective.

- Match → chat → message send/receive → realtime updates/read receipts as supported.
- Mobile nav → Likes/Matches/Chat/Profile accessible; desktop sidebar retains existing links.
- Settings, Privacy, Cafés, and Admin Panel remain unaffected.
- Offline/network failure/expired session/unauthorized action/empty deck/image failure states are honest and recoverable.

12. IMPLEMENTATION ORDER & CHANGE MANAGEMENT

■ Complete read-only reconciliation audit and identify exact code paths.

■ Create/update a concise implementation checklist and map requirements to files/components/backend contracts.

■ Implement Phase 1 and run its focused tests.

■ Implement Phase 2 and run its focused tests.

■ Implement Phase 3 and run its focused tests.

■ Implement Phase 4 and run its focused tests.

■ Implement Phase 5 and run responsive/accessibility/security checks.

■ Run the complete existing and newly created test suites, TypeScript, lint, build, compile_applet, and lint_applet.

■ Inspect the final diff for unrelated changes, branding drift, Admin Panel changes, accidental secrets, and generated artifacts.

■ Produce the final report. Do not commit, push, deploy, or run remote SQL/migrations.

Do not stop after each phase awaiting user approval. The requested scope is all five phases in this local workspace. However, do not force a risky implementation through a missing backend dependency; mark it blocked and explain.

13. REQUIRED ACCEPTANCE CRITERIA

- New members can register and verify email using the existing secure flow.
- New members can complete a guided, six-step, resumable onboarding experience.
- Existing complete members are not forced through onboarding; incomplete legacy members retain their data and can resume safely.
- Onboarding and Profile editor share consistent fields, validation, tags, photo behavior, and actual completeness calculation.
- Photos use the existing secure profile-photos pipeline and ownership controls.
- Discovery uses only real eligible profiles and provides multi-photo navigation, reliable Like/Pass controls, accessible buttons and honest states.
- Likes and Like back use real backend operations and resist duplicate actions.
- Matches are created only by the authoritative existing backend mutual-like process.
- Compatibility scores and explanations use real deterministic data and are not fabricated.
- Matched chat preserves existing history, realtime behavior, read receipts, typing, and participant authorization.
- Member navigation is usable on mobile and desktop without removing existing member areas.
- Settings, Privacy, Cafés, Admin Panel, existing branding, and unrelated features remain unchanged.
- No fake/demo data, no security weakening, no secret exposure, no production SQL/migrations, no remote data/settings changes, no push, and no deployment.
- All tests/build checks are reported accurately, with blockers and unverified items clearly separated.

14. REQUIRED FINAL REPORT — DO NOT OMIT

At completion, provide a detailed evidence-based report with these sections:

■ Executive summary: what was implemented and what remains blocked.

■ Initial audit reconciliation: which supplied audit findings were confirmed, corrected, or not verifiable.

■ Phase-by-phase matrix: each requirement marked Implemented / Partially implemented / Blocked / Not applicable, with evidence.

■ Exact changed files and a one-line explanation for each.

■ Routes/components/hooks/services/types/constants changed.

■ Backend contract reuse: tables, columns, RPCs, triggers, Edge Functions, Storage buckets, and RLS policies relied upon (names only; do not reveal secrets).

■ Local migration files created, why each is necessary, and why none was executed.

■ Security/privacy review and any backend blockers.

■ Test commands and exact results, including pass/fail counts.

■ TypeScript, lint, build, compile_applet and lint_applet results.

■ Responsive widths/devices actually checked; distinguish browser-verified checks from code inspection.

■ Manual scenarios actually executed versus scenarios not executable in the local environment.

■ Known issues, risks, assumptions, and recommended next actions.

■ Explicit confirmation: no remote SQL executed, no remote migrations applied, no production data/settings changed, no secrets changed, no Git push/commit, and no deployment.

15. STOP CONDITIONS — DO NOT CROSS THESE BOUNDARIES

• If the workspace does not contain the expected source files, do not invent a replacement project. Report what is missing.

• If a task requires remote credentials, production data access, or a live Supabase write, do not proceed. Mark it blocked.

• If a schema field or enum is uncertain, inspect source and migrations; do not guess. If still unresolved, do not add a speculative column or enum.

• If a UI requirement conflicts with existing product behavior or privacy rules, preserve the secure existing behavior and report the conflict.

• If tests fail, investigate and fix within scope; never remove assertions, skip tests, or report false success.

• If a requested change risks Admin Panel behavior or global branding, isolate it or leave it blocked rather than changing protected areas.

• Do not claim production readiness based solely on local build success. State clearly what was and was not verified.

FINAL DIRECTIVE

Implement the complete five-phase Datevra Tinder-style member experience in the existing local workspace, sequentially and safely, using real existing systems. Audit first, reuse before creating, preserve all protected areas, test every phase, and report honestly. The goal is a complete, reliable member experience—not a superficial visual imitation. No production operations, no SQL execution, no deployment, and no Git push.